

Five algorithms, one frontier — practised with the ai9414 demos.



About Yi

1st year MPhil student

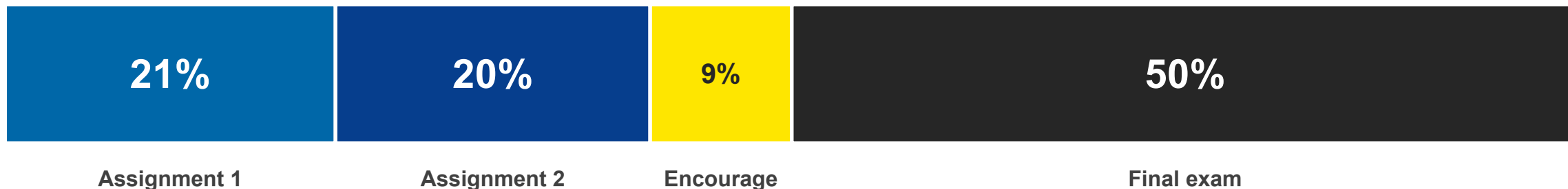
interesting in HRI, Computer Vision, 3DGS and VR

Studying in UNSW in 2025 as a coursework master(8543)

First Time to be a tutor

Contact: z5546082@ad.unsw.edu.au

● How the course is assessed



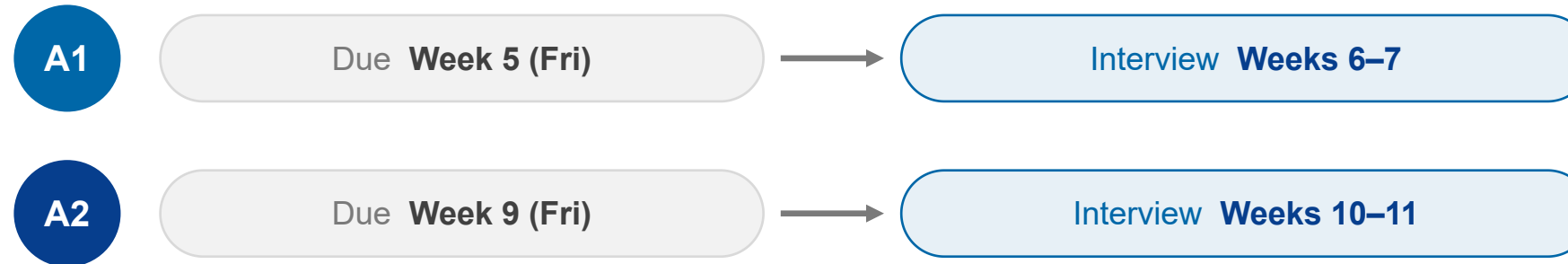
To pass

Score at least **40% on the final exam** and **≥ 50 overall**.

Encourage mark (9%)

Just turn up and code along — it includes the Week 1 quiz.

● Assignments: deadlines & interviews



In the interview

Book a slot on Moodle. You'll explain your code and make a small change — for example the learning rate or batch size. **You're marked on understanding, not on how the accuracy moves.**

Submission

Submit a Jupyter notebook `.ipynb` (a `.py` file may also be accepted).

● Tools, support & staying on track

Tools

- **Anaconda** — batteries-included, cross-platform
- **VS Code** — a solid editor
- **Google Colab** — run it in the browser

Projects are small — no paid compute needed.

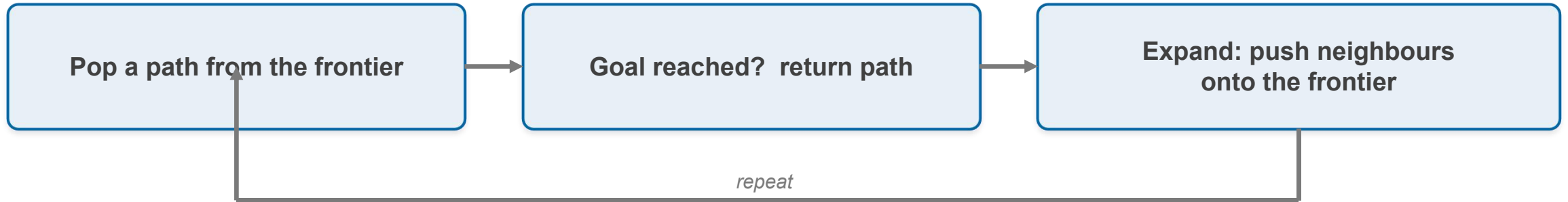
Getting help

- **Forum first** — tutors watch it and reply fast
- **Email** — as a backup
- **Week 11** — a review session runs even with no tutorial

Teams

- **Each weekend** — notes, slides and code are posted
- **Use them** — to revise for the final

● One search loop — five ways to order the frontier



The frontier is the protagonist

All five algorithms run the same loop. The only thing that changes is which path leaves the frontier next — that single rule decides the algorithm.

Uninformed (no heuristic) DFS • BFS • UCS

Order the frontier by structure, or by the cost paid so far.

Informed (uses a heuristic) Greedy • A*

Order the frontier using an estimate of the distance still to go.

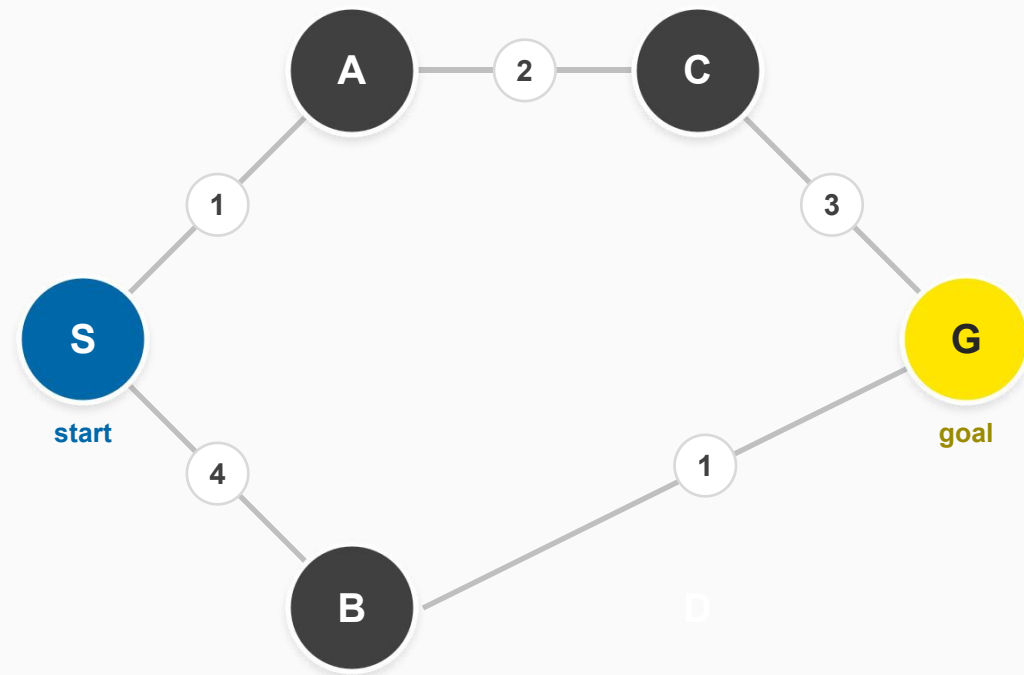
● A small weighted graph: reachability vs. path cost

DFS / BFS — unweighted

Reason about reachability and depth. Edge weights are ignored; what matters is how nodes connect and how deep they sit.

UCS / A* — weighted

Reason about path cost $g(n)$. Each edge has a price, so the cheapest route — not the shallowest — is the prize.



● Depth-First Search (DFS)

UNINFORMED

Follow one branch as far as it goes, then backtrack.
The most recently discovered node is always the next one explored.

DATA STRUCTURE

Stack, or recursion

PRIORITY RULE

Last in, first out (LIFO)

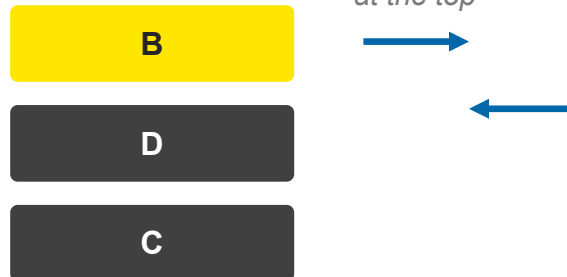
GUARANTEE

Complete on a finite graph
if repeated states are
handled — but the path it
finds need not be shortest.

RUN THE DEMO

ai9414 demo graph-dfs

FRONTIER: A STACK



Tip Neighbours are returned in sorted node-id order. Push them in reverse if you want the alphabetically-first neighbour expanded first.

● Breadth-First Search (BFS)

UNINFORMED

Expand all depth-0 nodes, then depth-1, and so on.
The earliest discovered node is always explored next.

DATA STRUCTURE

Queue (e.g.
`collections.deque`)

PRIORITY RULE

First in, first out (FIFO)

FRONTIER: A QUEUE

**GUARANTEE**

In an unweighted graph,
BFS finds a shallowest path
measured in number of
edges.

RUN THE DEMO

ai9414 demo
graph-bfs

Tip Because it sweeps level by level, the first time BFS reaches the goal it has used the fewest possible edges.

Two rules that trip people up

1 Re-entering the frontier

A node is already in the frontier, not yet visited — and you reach it again. May it go back in?

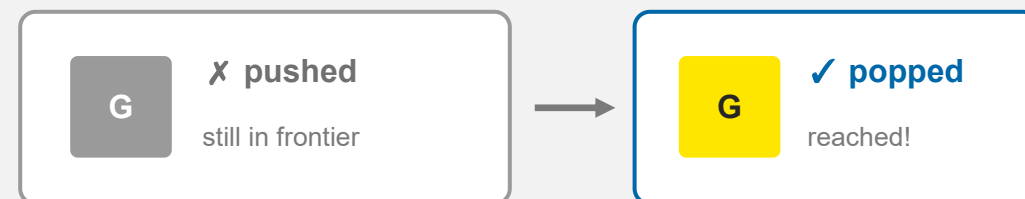
✓ **DFS — push it again.** A node counts as visited only when it is popped, so a later, different path may legitimately reach it.

✗ **BFS — skip it.** BFS marks a node the moment it is discovered, so each node enters the queue at most once.

Why? in the demos BFS marks neighbours visited inside `get_neighbours`; DFS only marks a node when the recursion actually reaches it.

2 When has the goal been reached?

Seeing the goal appear in the frontier is not enough — the search is not done yet.



Rule: a node — including the goal — is reached only when it is removed from the frontier (popped / expanded). Keep searching until the goal is popped; only then is the path final.

● Uniform-Cost Search (UCS)

UNINFORMED

Generalises BFS to positive edge costs. Instead of the oldest path, expand the frontier path with the lowest cost so far, $g(n)$.

DATA STRUCTURE

Priority queue (e.g. heapq)

PRIORITY RULE

Lowest $g(n)$ first

FRONTIER: PRIORITY QUEUE, ORDERED BY G

B $g=2$

→ *lowest g
leaves first*

D $g=5$

F $g=9$

GUARANTEE

With positive edge costs, UCS returns an optimal (lowest-cost) path to the goal.

RUN THE DEMO

ai9414 demo
graph-ucs

Tip Keep a best-cost dictionary: a later route to a node is ignored unless it beats the best cost already known for that node.

● Heuristics: a straight-line guess to the goal

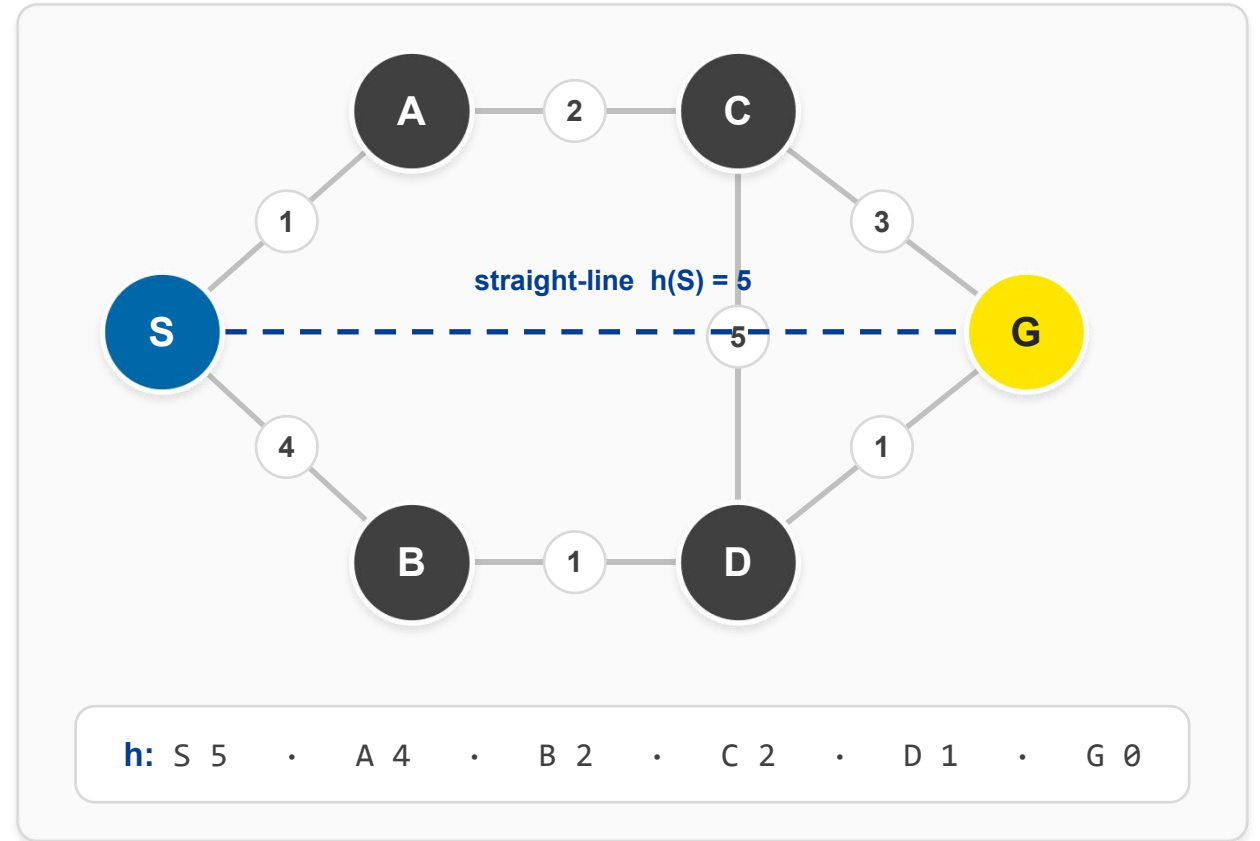
$h(n)$ — the straight-line distance

How far node n sits from the goal as the crow flies. It is looked up from a table and is usually different from the real, along-the-edges path cost.

Admissible

h never overestimates the true remaining cost. An admissible heuristic is what keeps A^* optimal.

Uninformed search (DFS / BFS / UCS) uses no heuristic. Greedy and A^ do.*



● Greedy / Best-First Search

INFORMED

Always expand the node that looks closest to the goal — the lowest heuristic $h(n)$. It ignores the cost already paid to get there.

DATA STRUCTURE

Priority queue (ordered by h)

PRIORITY RULE

Lowest $h(n)$ first

FRONTIER: PRIORITY QUEUE, ORDERED BY h

C $h=2$

→ *lowest h
leaves first*

A $h=4$

B $h=5$

GUARANTEE

Fast and often effective, but greedy — it is neither complete nor optimal in general.

WATCH OUT

Looks at $h(n)$ only — a cheap-looking node can lead down an expensive path.

Tip A* fixes this by adding the cost so far $g(n)$ back in: $f = g + h$. Greedy is just A* with the g term switched off.

● A* Search

A* = UCS + greedy. Use the cost so far $g(n)$ and a heuristic $h(n)$, and expand the path with the lowest $f(n)$.

FRONTIER: PRIORITY QUEUE, ORDERED BY $F = G + H$

C $f=1.9$

→ *lowest f
leaves first*

N $f=2.6$

H $f=3.0$

$$f(n) = g(n) + h(n)$$

INFORMED

DATA STRUCTURE

Priority queue

PRIORITY RULE

Lowest $g(n) + h(n)$ first

GUARANTEE

If the heuristic is admissible (never overestimates), A* returns an optimal path.

RUN THE DEMO

ai9414 demo
graph-astar

Tip heuristic_to_goal gives the straight-line distance — admissible for the supplied spatial graphs.

● The five algorithms at a glance

Algorithm	Frontier discipline	Uses heuristic?	Optimal path?
DFS	Stack (LIFO)	No	No
BFS	Queue (FIFO)	No	Fewest edges only
UCS	Priority by g	No	Yes — lowest cost
Greedy	Priority by h	Yes	No
A*	Priority by g + h	Yes	Yes — with admissible h

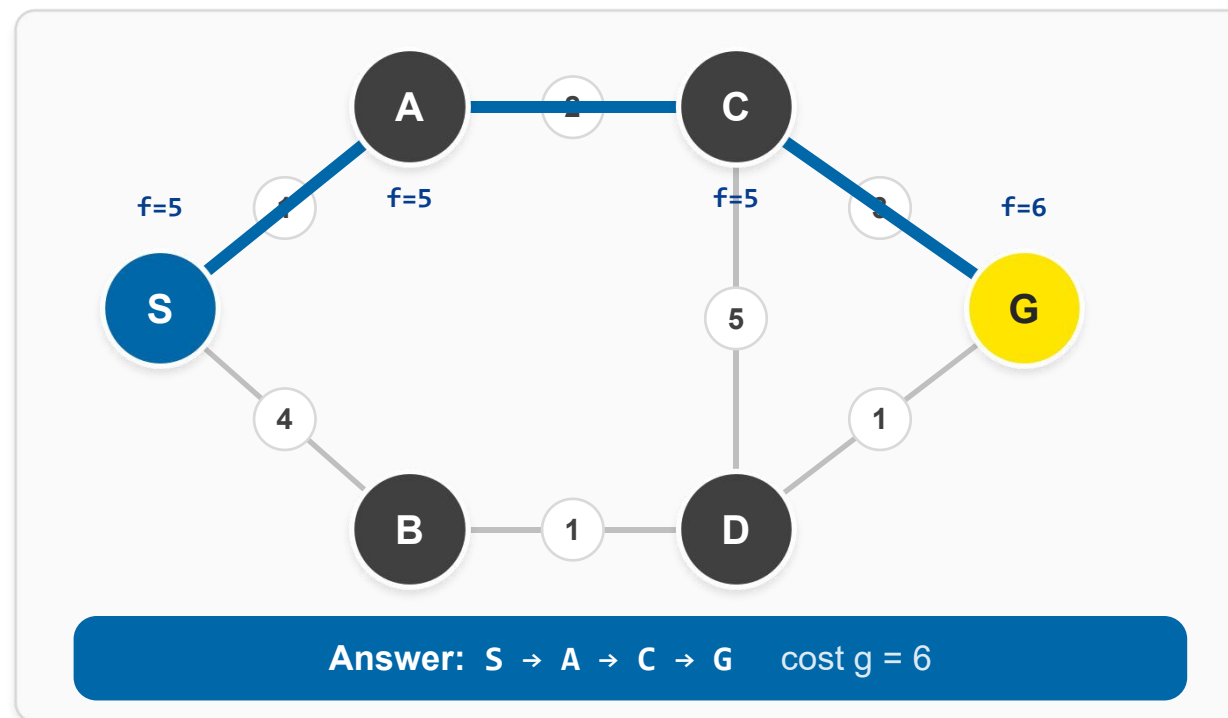
Two splits: **uninformed** (DFS, BFS, UCS) reason without an estimate; **informed** (Greedy, A*) use a heuristic. Only UCS and A* guarantee the cheapest path.

EXAM METHOD

Worked example: A* the way the exam wants it

HOW TO ANSWER IT

- 1 List the frontier with each path's priority $f = g + h$.
- 2 Expand (pop) the lowest- f path each time.
- 3 Met a node again? Keep the cheaper g , drop the rest.
- 4 Stop only when the goal is popped — not when added.
- 5 Recover the path via parent pointers, then reverse.



On the exam a 2-hour multiple-choice paper (~12–14 questions) tests calculation, not code — so hand-trace on scrap paper and pick the right path. Nothing is submitted.

● Set-ExecutionPolicy -ExecutionPolicy RemoteSigned - Scope CurrentUser

1 · Create a Python 3.11+ environment and install the package

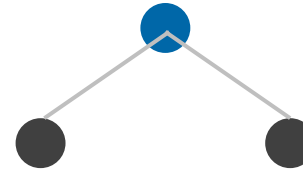
```
python3 -m venv .venv
Py -m venv .venv # for windows
source .venv/bin/activate
# Windows: .\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install ai9414
```

2 · Check the CLI, then launch each demo

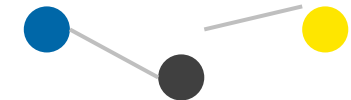
```
ai9414 list # or: python -m ai9414 list
ai9414 demo graph-dfs --example small
ai9414 demo graph-bfs --example small
ai9414 demo graph-ucs --example small
ai9414 demo graph-astar --example small
```

Each demo opens a browser with two panels

Search Tree



Spatial Graph



Left the search tree — the frontier history.

Right the fixed graph — active route + explored edges highlighted.

Leave the command running while you use the demo; stop with Ctrl-C.

TWO MODES

● Playback vs. live Python

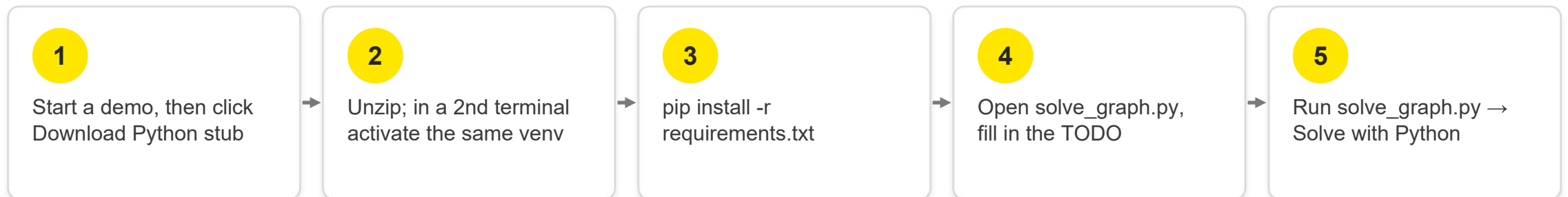
Playback mode

The supplied reference solver drives the visualisation — perfect for seeing how an algorithm is supposed to behave before you code.

Live Python mode

Your local Python receives the graph and returns a trace; the browser replays exactly what your code produced.

THE LIVE-PYTHON WORKFLOW



THE CONTRACT

● What your solver returns

Result dictionary (UCS / A* — the superset)

```
{
  "algorithm": "astar",    # dfs | bfs | ucs | astar
  "status": "found",      # or "not_found"
  "trace": trace,
  "path": path,
  "best_cost": best_cost, # UCS / A* only
  "visited_order": visited_order,
}
```

Trace events by algorithm

DFS

start · expand · backtrack · found · fail

BFS

start · expand · found · fail

UCS / A*start
t
·
expand · consider_edge · relax · found · fail

Every entry needs `step`, `action`, `node`, `parent`, `depth`, plus algorithm-specific fields (`stack` / `route` / `path_cost` / `heuristic` / `priority` ...). Use `None` for optional fields.

Counting convention DFS / BFS report nodes reached (marked when first discovered). · UCS / A* report nodes expanded (popped from the priority queue). State your convention if you differ.

● Helpers, heaps, and the best-cost trick

Inspect neighbours via the supplied helper

```
# DFS / BFS – unweighted
from ai9414.graph_bfs import get_neighbours
for nb in get_neighbours(graph, node): ...

# UCS / A* – neighbours carry edge cost
from ai9414.graph_ucs import get_neighbours
for nb, cost in get_neighbours(graph, node): ...
```

Priority queues with heapq

```
import heapq
frontier = []
# UCS: priority = new path cost
heapq.heappush(frontier, (new_cost, nb))

# A*: add the heuristic estimate
pri = new_cost + heuristic_to_goal(graph, nb)
heapq.heappush(frontier, (pri, new_cost, nb))
```

Keep a best_costs dictionary (UCS & A*). A later route to the same node should be ignored unless it improves the best known cost to that node — this is what stops the search from re-expanding worse paths.

YOUR MISSION

• Eight steps to complete

1

Environment check

Install ai9414, run list, launch all four demos, record each start & goal.

3

BFS solver

Implement solve_bfs; compare path length with DFS and explain the shallow guarantee.

5

A* solver

Implement solve_astar with heuristic_to_goal; compare expanded nodes with UCS.

7

Heuristic experiment

Set the heuristic to zero, rerun, identify the behaviour — then restore it.

2

DFS solver

Implement solve_dfs; submit path, visited_order, and where DFS backtracks.

4

UCS solver

Implement solve_ucs with heapq; report best_cost and the first five expanded nodes.

6

Large-example comparison

Tabulate path, cost, #expanded, optimality for BFS/UCS/A*; add a 4-algorithm summary.

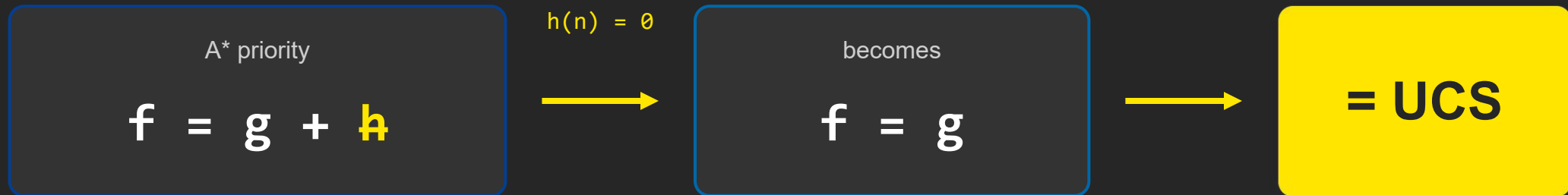
8

Reflection

Explain why the side-by-side tree and graph views aid understanding (2–3 sentences).

Set the heuristic to zero

What does A* turn into?



With no estimate to the goal, A* orders the frontier purely by cost so far — exactly uniform-cost search. Restore the original heuristic before submission.